Precision-Aware Optimizer

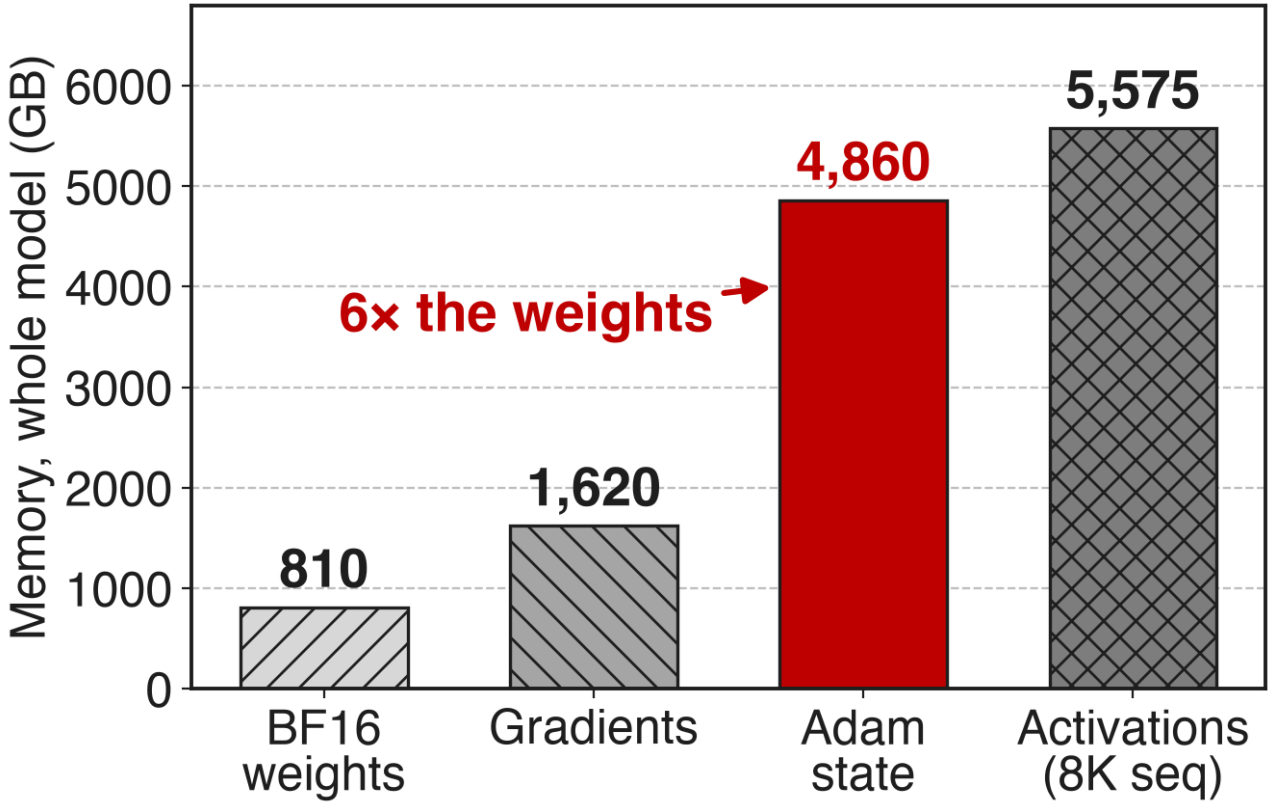
Store Adam state in BF16 or FP8, update in FP32

NVIDIA · Megatron-Core (BF16 states since v0.13) + Transformer Engine FusedAdam

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Adam's state outweighs the model six to one

- Mixed-precision Adam: FP32 master, m and v
- **12 B** per parameter vs **2 B** for the BF16 weight
- Llama-405B: **4,860 GB** of Adam state



Optimizer state takes 32 GB per GPU

32.1 GB

of FP32 master weights and Adam moments per GPU for DeepSeek-V3 (BF16, 256 GPUs): 16% of a 199.5 GB footprint on an 80 GB H100

Megatron-Core MoE report, arXiv 2603.07685, Table 3 and §4.1.6

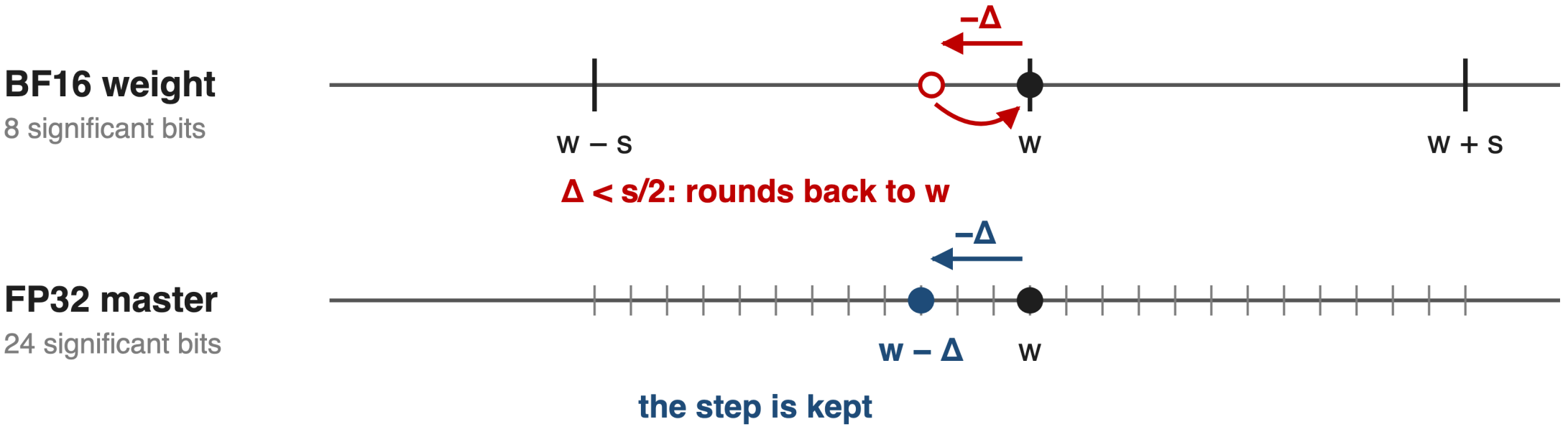
Sharding and offloading keep all 12 bytes

- **Shard** (distributed optimizer, FSDP)
- 12/d per rank, still **12 B** in total
- Needs data-parallel ranks (deck 70)

- **Offload** to CPU memory
- Saves 15–20 GB on DeepSeek-V3
- **+0.1–0.2 s** per iteration, needs a fast host link

Neither makes the 12 bytes per parameter any smaller

Naive BF16 Adam rounds small updates away



Schematic. BF16 keeps 8 significant bits: a step below half a spacing (at most 2^{-8} of $|w|$) is lost

The villain: storing state in the same precision the update is computed in

Insight: store compactly, compute in FP32

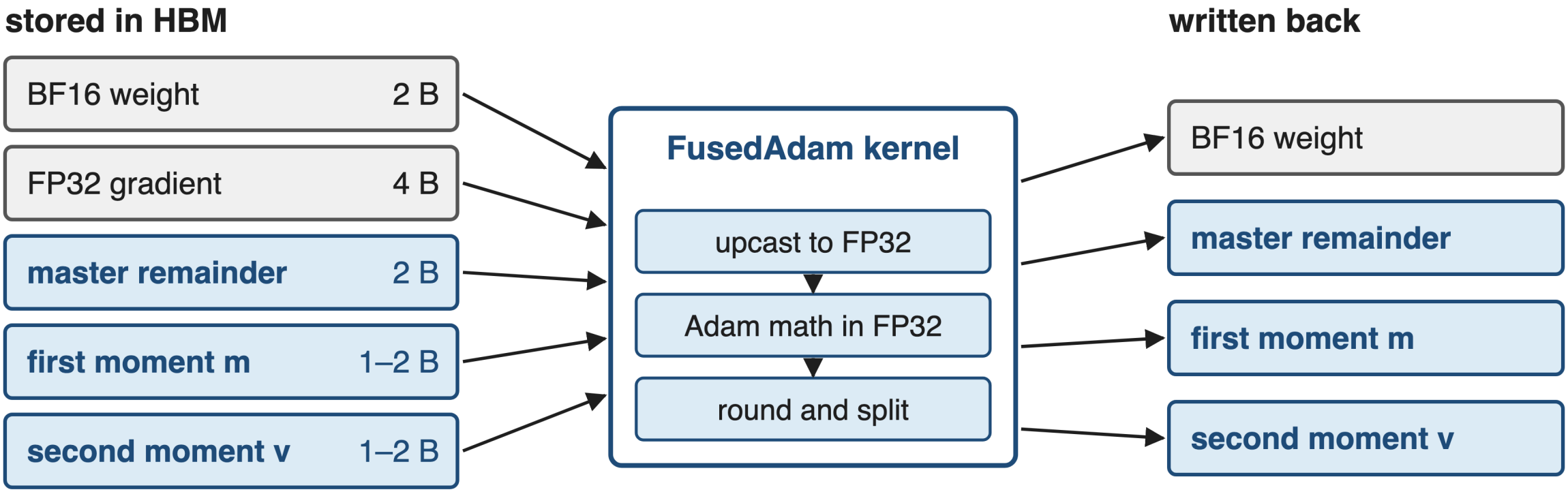
- Moments only feed the update: BF16 storage suffices
- The FP32 master's top half already is the BF16 weight

Insight: store compactly, compute in FP32

- Moments only feed the update: BF16 storage suffices
- The FP32 master's top half already is the BF16 weight

Decouple storage from compute: upcast inside the fused kernel, update in FP32, round once on the way out

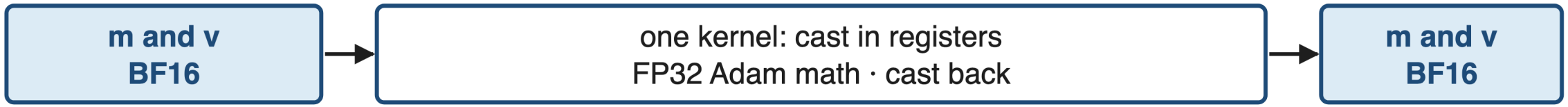
FusedAdam upcasts, updates, and rounds once



--use-precision-aware-optimizer --exp-avg-dtype / --exp-avg-sq-dtype {fp32, fp16, bf16, fp8} --main-params-dtype {fp32, fp16} --main-grads-dtype {fp32, bf16}

❶ Moments travel in BF16 or FP8

BF16 moments: fused



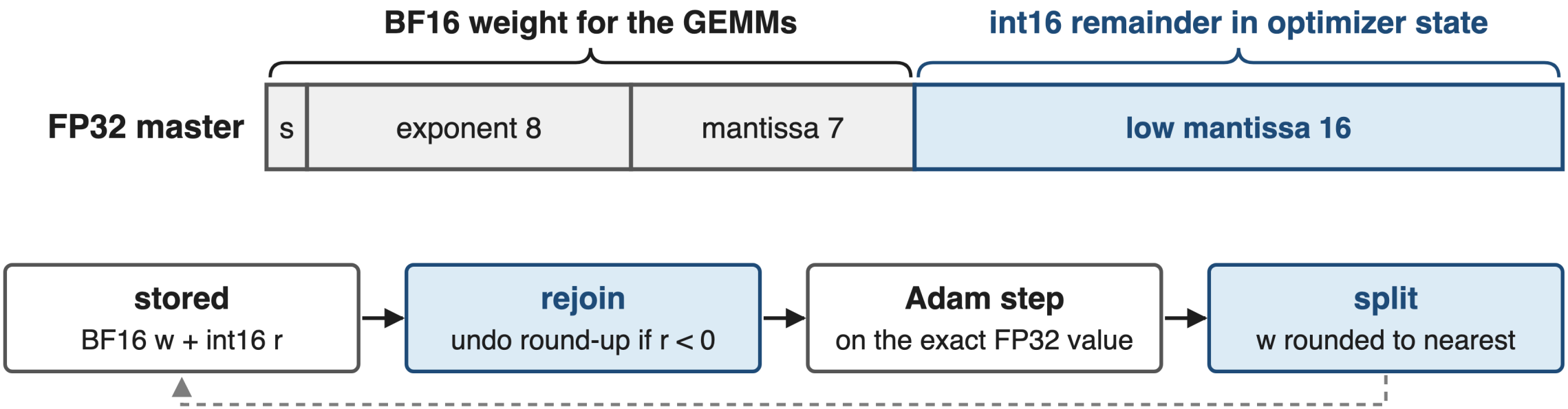
FP8 or FP16 moments: scaled per tensor



scale = absmax / 448 for FP8 E4M3

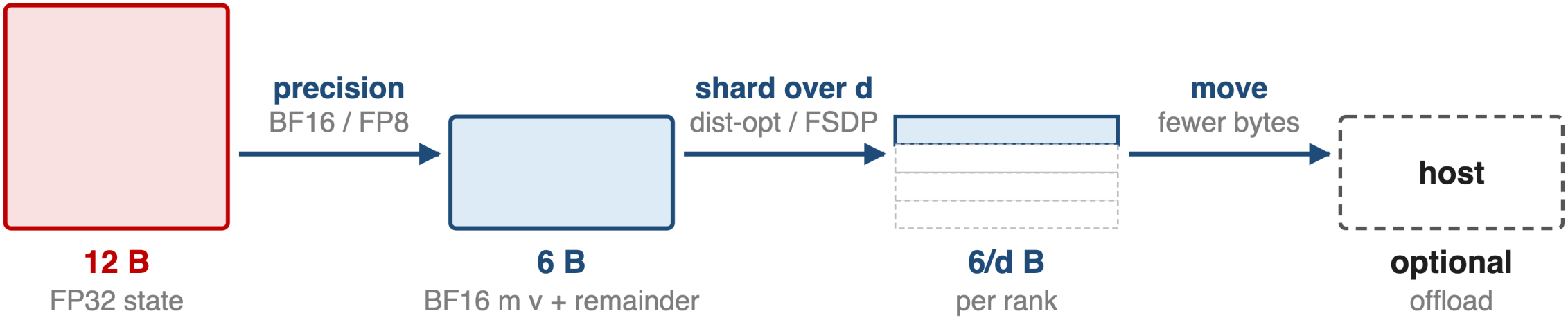
BF16 costs nothing extra; FP8 and FP16 share one FP32 scale per tensor

❷ BF16 weight plus remainder is exact FP32



The master weight costs 2 bytes instead of 4 and stays bit-exact FP32

❸ Smaller state shards and offloads cheaper

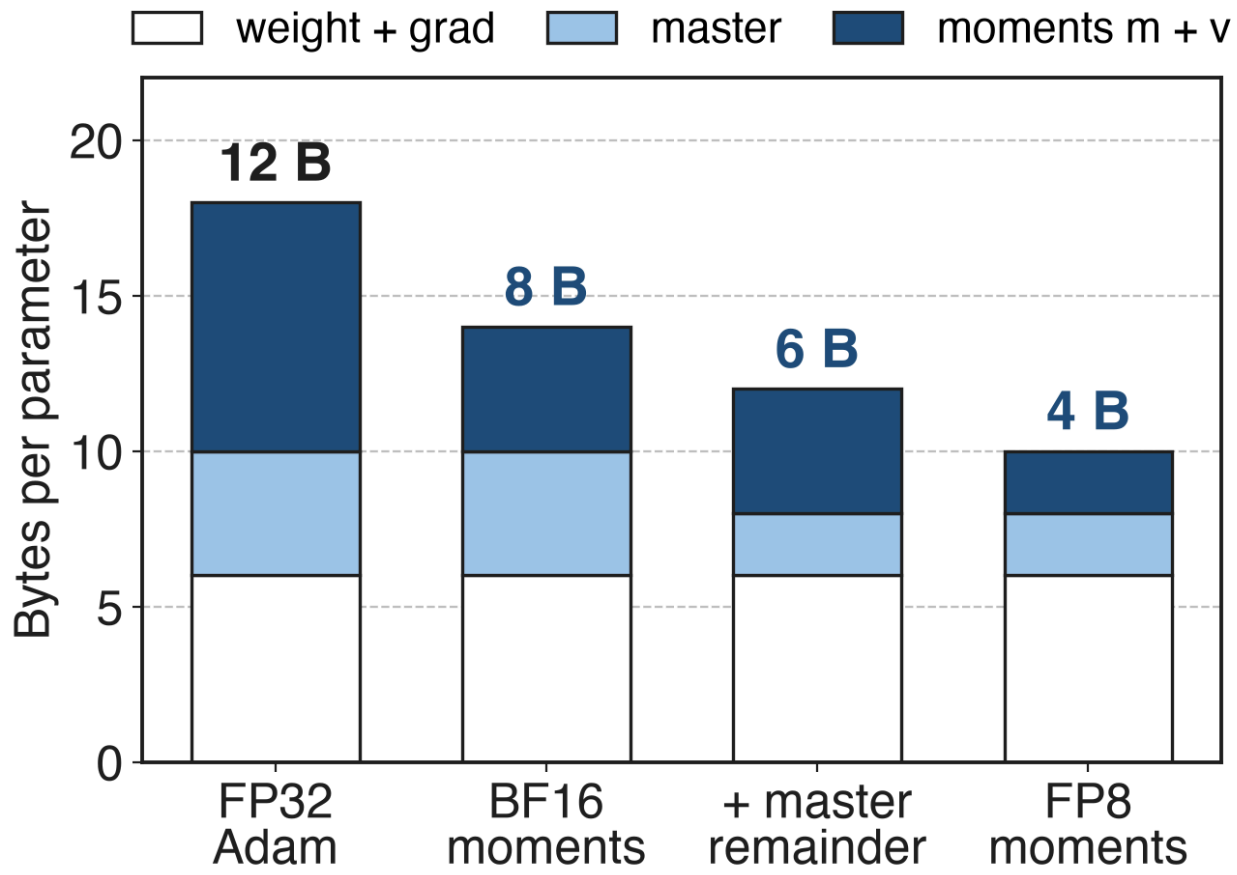


Optimizer-state bytes per parameter with BF16 moments + remainder (the MCore report's 6 + 8/d keeps a full FP32 master)

Distributed optimizer: 6 + 12/d → 6 + 8/d bytes per rank; CPU offload requires this flag; Megatron-FSDP takes the same dtypes

Optimizer state drops from 12 to 6 bytes

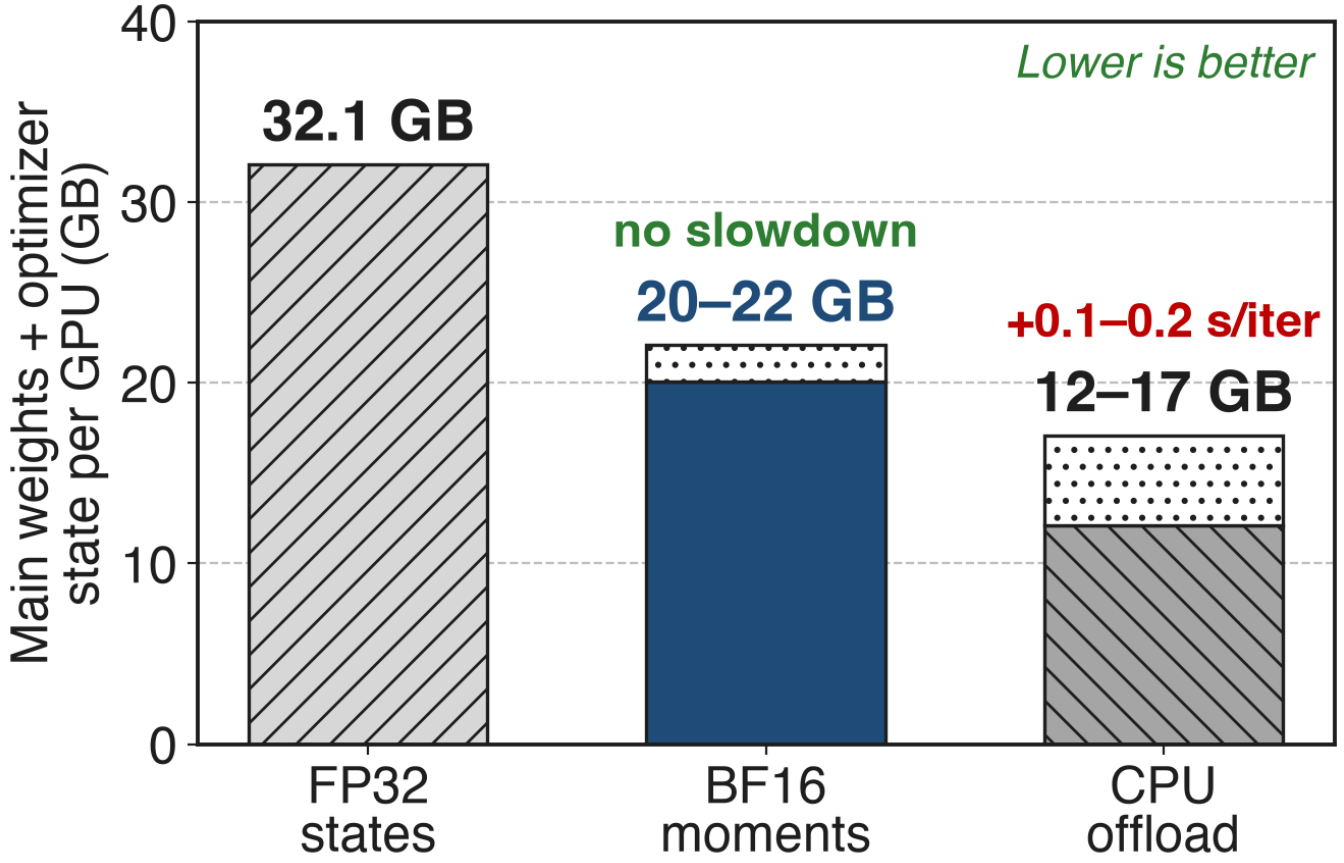
- BF16 moments: **12 → 8 B**
- + master remainder: **6 B**, still exact FP32
- FP8 moments: **4 B**



Analytic, per parameter, no sharding; number on top = optimizer-state bytes (master + m + v)

BF16 moments free 10–12 GB per GPU

- DeepSeek-V3, 256 GPUs, BF16
- **No performance overhead**: cast inside the fused kernel
- Offload saves more but costs time



DeepSeek-V3 recipes keep low-precision states

- **MCore final configs**
- H100: **368** TFLOPS/GPU, GB200: **1,048**
- Both list low-precision optimizer states

- **DeepSeek-V3 training**
- BF16 moments, no observable degradation
- Master weights and gradients stay FP32

The cost: Adam only, one scale per tensor

- Adam via TE FusedAdam; remainders need TE ≥ 2.1
- CUDA graphs: FP32 or BF16 moments, no remainders
- FP8 and FP16: one scale per tensor, FP32 temporaries
- **FP8 moments**: no convergence study in these sources
- FP16 master is lossy

Summary

- Adam state: **12 B** per parameter; **32.1 GB/GPU** for DeepSeek-V3
- **Insight**: store compactly, compute in FP32 inside the fused kernel
- BF16/FP8 moments, exact FP32 master from a 16-bit remainder
- **12 → 6 B** per parameter; **10–12 GB/GPU** saved, no slowdown

https://github.com/NVIDIA/Megatron-LM